

10.3. Переопределение операций

*«Мы говорим Ленин – подразумеваем Партия,
Мы говорим Партия – подразумеваем Ленин».
Владимир Маяковский.*

Одна из целей «эпизодического» ООП – создание необходимых пользователю форм представления (типов данных) в виде классов. Естественное желание, сделать их неотличимыми до такой степени, чтобы с ними можно было работать как с обычными переменными базовых типов. **Переопределение операций** обеспечивает перепрограммирование операций таким образом, что в качестве операндов в них могут использоваться объекты интересующего нас типа.

Замечание: в Си++ обычно используется термин «переопределение операторов», более того, он закреплён синтаксически. Но коль скоро для обозначения таких действий мы использовали термин «операции» (см. 1.4), то будем до конца последовательны.

Итак, переопределение операций заключается в том, что транслятор «начинает понимать», что означает известная операция, если одним (или всеми) его операндами является объект нужного нам класса. Прежде всего, определимся с правилами такого переопределения, что можно менять, а что нельзя:

- нельзя менять синтаксис языка – количество операндов, приоритеты операций и направление их выполнения;
- переопределение операций производится отдельно для каждого сочетания операндов, перестановка операндов транслятором не производится. Например операции с сочетаниями операндов **string+char[], char[]+string** – это различные операции;
- можно менять способы передачи операндов (по ссылке, по значению), тип и способ возвращения результата;
- никаких ограничений не накладывается на действия, выполняемые над объектами в переопределяемой операции (интерпретация операции может быть любой).

Переопределение операций внутри класса

Естественно, если операция работает с операндами, принадлежащими к некоторому классу, то её желательно внести в этот класс хотя бы для того, чтобы не было проблем с доступом к закрытой части объекта. Такой способ называется **переопределением операции в классе** и возможен, если первый операнд операции является объектом этого класса. Для этой цели вводится специально поименованный метод со следующим синтаксисом:

- метод определяется в классе первого операнда;
- имя метода – **operator<знак операции>;**
- первый операнд – текущий объект класса;
- второй операнд – формальный параметр, который может быть передан как по значению, так и по ссылке. Тип формального параметра должен совпадать с типом второго операнда;
- результат операции может быть произвольного типа, он может возвращаться как указатель, ссылка или значение;

- на действия, выполняемые в теле метода, ограничений не накладывается (содержательная интерпретация операции может быть любой);
- если формальный параметр или результат передаются по значению, а объект содержит динамические данные, то в классе необходим конструктор копирования, который автоматически вызывается при передаче такого операнда и возвращении результата по **return**.

Продолжим усовершенствование класса полинома, введя в него набор арифметических и логических операций с целью сделать его похожим на базовый тип. Параллельно обсудим ряд общих вопросов, касающихся переопределения.

Тип результата операции и способ его формирования может быть любым, а интерпретация – сколь угодно экзотической. Следить нужно только за соблюдением закрытости данных объекта и за корректностью работы с динамическими данными. Например, можно переопределить операцию [] таким образом, что она будет возвращать ссылку на коэффициент полинома, аналогично методу **get** (см. выше).

```
//-----103-01.cpp
// переопределение [] - ссылка на коэффициент
double &operator[](int k){ return get(k); }
```

При переопределении арифметических операций над объектами обычно используется **конвейер значений** – схема передачи параметров и результата по значению, обеспечивающая получение результата в виде «нового» временного объекта-значения при неизменности входных операндов. С этой целью операнды передаются как параметры-значения, либо при выполнении операции создаются локальные объекты-копии. Результат также возвращается по значению. В этой схеме явно и неявно используются конструкторы копирования, а количество промежуточных объектов может быть довольно большим (что следует учитывать при оценке эффективности программы).

```
//-----103-01.cpp
// переопределение операций
poly operator+(poly T){           // Переопределение сложения - конвейер значений
    T.add(*this);                 // Второй операнд по значению (копия)
    return T;                    // Добавление первого к копии второго
}
poly &operator-(poly T){          // Переопределение вычитания - конвейер значений
    for (int i=0;i<=T.n;i++)      // Второй операнд по значению (копия)
        T.pd[i]=-T.pd[i];       // Инвертировать коэффициенты копии второго операнда
    T.add(*this);                 // Копия второго + первый
    return T; }
poly operator*(poly &T){          // Умножение - конвейер значений
    poly R(n+T.n+1);             // Вспомогательный объект - сумма размерностей
    for (int i=0;i<=n;i++)        // Добавление частичных произведений всех пар
        for (int j=0;j<=T.n;j++)
            R.pd[i+j]+=pd[i]*T.pd[j];
    return R;                    // Возврат локального объекта по значению
}
poly operator*(int v){            // Умножение на целое - конвейер значений
    poly R(*this);               // Копия текущего – первого операнда
    for (int i=0;i<=R.n;i++)
        R.pd[i]*=v;
    return R; }
```

В качестве операндов можно использовать не обязательно объекты, но и другие, экзотические формы представления данных этого типа. Например, если коэффициенты полинома хранить в массиве, начиная с первого элемента, а в нулевом хранить размерность полинома, то можно переопределить сложение, создав внутри

```
//-----103-01.cpp
poly operator+(double *p){      // Переопределение сложения с массивом
    poly R((int)*p,p+1);        // Создать объект из массива
    return *this+R;              // p[0]-размерность, p[1]...p[n+1] - коэффициенты
}
```

Обратите внимание на синтаксис ***this+R** для вызова переопределенной ранее операции сложения вида **poly+poly** текущего объекта с вновь созданным. В Си++ в операциях можно однократно использовать безымянные объекты классов, записав имя класса с фактическими параметрами конструктора в скобках. Тогда переопределение можно сделать еще проще:

```
//-----103-01.cpp
poly operator+(double *p){      // Переопределение сложения с массивом
    return *this+poly((int)*p,p+1); // Безымянный объект
}
```

Переопределение операций сравнения имеет стандартную интерпретацию, вызывается внутреннего метода сравнения **compare** с возвратом логического значения.

```
// Переопределение сравнений
int operator<(poly &T) { return compare(T)<0; }
int operator<=(poly &T){ return compare(T)<=0; }
int operator>(poly &T) { return compare(T)>0; }
int operator>=(poly &T){ return compare(T)>=0; }
int operator==(poly &T){ return compare(T)==0; }
int operator!=(poly &T){ return compare(T)!=0; }
```

Особенности переопределения некоторых операций

Переопределение операции присваивания. Стандартная интерпретация присваивания предполагает выполнения следующих действий:

- разрушение содержимого текущего объекта – левого операнда (аналогично деструктору);
- копирование содержимого объекта-параметра (правого операнда) в текущий объект (аналогично конструктору копирования);
- возвращение ссылки на текущий объект.

```
//-----103-01.cpp

poly &operator=(poly &R){      // Присваивание
    if (&R==this) return *this; // Присваивание "сам в себя"
    delete []pd;               // Разрушить левую часть (текущий)
    load(R.n,R.pd);             // Копия правой части (аналог КК)
    return *this; }             // Возвращает ссылку на левый

poly &operator=(double *d){     // Экзотическое присваивание массива со счетчиком
    delete []pd;               // Разрушить левую часть (текущий)
    int nn=d[0];               // В начале массива - размерность
    load(nn,d+1);              // дальше - данные
    return *this; }            // Возвращает ссылку на левый
```

При переопределении присваивания в правой части может быть операнд любого типа, необходимо только подобрать ему достойную интерпретацию. Например, при присваивании полиному вещественного массива из первого его элемента извлекается размерность, а затем – сами коэффициенты.

Переопределение операции приведения типа. Особенность операции – отсутствие формальных параметров и спецификации типа результата, поскольку он и так

определяется приводимым типом. Переопределенная таким образом операция будет неявно вызываться всякий раз при присваивании целому числу значения объекта, либо при явном приведении объекта к этому типу. Содержательная интерпретация преобразования может быть связана с получением какой-либо численной характеристики объекта, например, размерности полинома.

```
//-----103-01.cpp
// переопределение приведения к int - возвращает размерность
operator int(){ return n; }
```

«Преобразовать» объект можно и к указателю. Если указатель интерпретировать как динамический массив, то можно, например, выгрузить в него внутренние данные объекта (например, коэффициенты полинома).

```
//-----103-01.cpp
// Переопределение приведения к double* -
// возвращение динамического массива
operator double*(){
    double *q=new double[n+2];
    q[0]=n; // в нулевой ячейке – размерность полинома
    for (int i=0;i<=n;i++) // начиная с первой - коэффициенты
        q[i+1]=pd[i];
    return q;}

```

Такое преобразование будет срабатывать в том числе и при присваивании объекта-полинома указателю (выражение вида **double *q=a1;**).

Переопределение операций () и []. Переопределение операции **()** позволяет использовать синтаксис вызова функции применительно к объекту класса (имя объекта с круглыми скобками). Количество операндов в скобках может быть любым. Переопределение операции **[]** позволяет использовать синтаксис элемента массива (имя объекта с квадратными скобками). В классе полиномов выражение вида **a[i]** позволяет получить ссылку на i-ый коэффициент полинома, а выражение вида **a(i,v)** записать значение **v** в качестве i-го коэффициента.

```
//-----103-01.cpp
// переопределение [] - ссылка на коэффициент
double &operator[](int k){ return get(k); }
// переопределение () с двумя параметрами - запись коэффициента
poly &operator()(int k, double v){
    get(k)=v;
    return *this;}

```

```
//-----103-01.cpp
poly operator++(int){ // Переопределение poly++
    poly T(*this); pd[0]++; return T; }
poly operator++(){ // Переопределение ++poly
    pd[0]++; poly T(*this); return T; }

```

Переопределение операций new и delete. Операции создания и уничтожения объектов в динамической памяти могут быть переопределены следующим образом

```
static void *operator new(size_t size);
static void operator delete (void *);
```

где **void *** - указатель на область памяти, выделяемую под объект, **size** - размер объекта в байтах, **size_t** - тип размерности области памяти, **int** или **long**. Переопределение этих операций позволяет написать собственное распределение памяти для объектов класса.

Переопределенные операции будут вызываться при создании динамических объектов (но не их массивов). Естественно, если мы разработаем собственную систему динамического распределения памяти (ДРП), то она должна использоваться и для динамических данных самих объектов.

Попробуем приспособить простую ДРП (см. 9.2) для создания динамических объектов класса полиномов. Для этого нужно превратить набор функций в класс, а их общие данные – в данные объекта.

```
//-----103-02.cpp
// Переопределение операций распределения памяти
class memory{
    char *pa;
    int sz0;
public: memory(int sz)      {...}
    ~memory(){ delete []pa; }
    void *malloc(int sz)   {...}
    void show()            {...}
    void free(void *q0)    {...}};
```

Затем в класс степенного полинома нужно добавить статический объект класса **memory** (общий для всего класса) и использовать его методы в переопределенных операциях **new** и **delete**, а также для создания и уничтожения собственных динамических массивов коэффициентов в объектах класса **poly**.

```
//-----103-02.cpp
// Класс степенного полинома с собственным распределением памяти
class poly{
    int n; // степень полинома
    double *pd; // динамический массив коэффициентов
void load(int n0, double p[]){
    n=n0; // закрытый метод загрузки массива
    pd=(double*)MEM.malloc(sizeof(double)*(n+1));
    for(int i=0;i<=n;i++)
        pd[i]=p[i]; }
void extend(int n1){ // увеличение размерности полинома
    if (n1<=n) return;
    double *pd1=(double*)MEM.malloc(sizeof(double)*(n1+1));
    for (int i=0;i<=n;i++) pd1[i]=pd[i];
    for (;i<=n1;i++) pd1[i]=0;
    n=n1;
    MEM.free(pd); // удалить старый массив
    pd=pd1; // считать новый за старый
}
void normalize(){ // нормализация - удаление лишних 0
    while(n>0 && pd[n]==0) n--;
}
public:
    static memory MEM; // статический объект - ДРП
    poly(){ // "пустой" полином - нулевой степени
        n=0; // с нулевым коэффициентом
        pd=(double*)MEM.malloc(sizeof(double));
        pd[0]=0;}
    poly(int m){ // полином заданной степени
        n=m; // с нулевыми коэффициентами
        pd=(double*)MEM.malloc(sizeof(double)*(n+1));
        for(int i=0;i<=n;i++)
            pd[i]=0; }
    poly(int n0,double p[]){load(n0,p); } // конструктор из массива коэффициентов
    poly(poly &T){ load(T.n, T.pd); } // конструктор "объект из объекта"
    ~poly(){ MEM.free(pd); } // деструктор
```

```
//-----
// переопределение операторов new и delete в классе
static void *operator new(size_t sz){ return MEM.malloc(sz); }
static void operator delete(void *p){ MEM.free(p); }
//-----
};
memory poly::MEM(10000); // Определение статического элемента класса
void main(){
    double A[]={1,-2,3,-4},B[]={5,3,6};
    poly a1(3,A),a2(2,B),*p;
    poly::MEM.show(); // Явный вызов метода в статическом элементе
    p=new poly(3,A);
    poly::MEM.show();
    delete p; }
```

Аналогичные замены нужно сделать и в переопределяемых операциях: присваивании и вводе из потока.

Переопределение операции вне класса

Бывают случаи, когда переопределить операцию внутри класса не удастся:

- первый операнд является базовым типом, например, переопределение операции с сочетанием операндов **int*poly**;
- первый операнд (текущий объект) требуется передать по значению (а не через указатель);
- класс первого операнда недоступен, т.е. уже написан и оттранслирован.

Во всех случаях на помощь приходит способ переопределения, в котором принадлежность к классу отсутствует вовсе, а все операнды передаются явно через формальные параметры. Такая функция существует сама про себе, вне класса и имеет следующие особенности:

- имя функции – **operator<знак операции>**;
- первый и второй операнды – формальные параметры, могут быть переданы как по значению, так и по ссылке. Типы формальных параметров должны совпадать с типами операндов;
- если функция-оператор должна иметь доступ к закрытым данным операнда, то она должна быть дружественной в классе этого операнда.

Переопределять операцию вида **int*poly** приходится, поскольку первый операнд является не классом, а базовым типом. Схема передачи параметров - конвейер значений, второй формальный параметр (второй операнд) – полином – передается по значению, а затем его коэффициенты умножаются на целое. Поскольку на приходится работать с «внутренностями» объекта **poly**, то этот оператор объявляется дружественным в классе **poly** (если быть более точным, в нашем примере он *одновременно* объявляется дружественным в заголовке класса **poly** и тут же определяется, хотя формально к классу не относится).

```
//-----103-01.cpp
// переопределение операции int*poly - конвейер значений
friend poly operator*(int v, poly R){
    for (int i=0;i<=R.n;i++)
        R.pd[i]*=v;
    return R; }
```

Наиболее известный пример использования дружественного переопределения вне класса – переопределение ввода и вывода в стандартные потоки **istream** и **ostream**. Операции вида **istream>>poly** и **ostream<<poly** переопределяются вне класса, поскольку первым операндом являются объекты, недоступные для программирования. Эти операторы должны быть дружественными в классе **poly**, поскольку используют данные этого класса.

Обе операции используют схему передачи параметров – **конвейер ссылок**. Они возвращают в качестве результата ссылку на первый операнд – поток, что позволяет выполнять несколько операций << или >> в цепочке: ссылка на объект – поток будет передаваться по конвейеру.

Другая важная особенность переопределения – сохранение объекта в потоке в **саморазворачивающемся формате** с учетом его переменной размерности. С этой целью в поток сначала записывается размерность полинома, а затем его коэффициенты. Форма представления объекта исключает всякую «красивость»: все-таки поток – это не столько средство ведения диалога, сколько возможность сохранять объекты в текстовых файлах.

И наконец, не надо забывать, что при чтении объекта из потока происходит разрушение его старого содержимого. Для этого в нем выполняются действия, аналогичные деструктору, а память при чтении выделяется «по новой».

```
//-----103-01.cpp
// переопределение вывода в поток - дружественный оператор
// конвейер ссылок
friend ostream &operator<<(ostream &O, poly &T){
    O << T.n << endl;
    for (int i=0;i<=T.n; i++)
        O << T.pd[i] << " ";
    O << endl;
    return O; }
// переопределение ввода из потока - дружественный оператор
// конвейер ссылок
friend istream &operator>>(istream &O, poly &T){
    delete []T.pd;
    O >> T.n;
    T.pd=new double[T.n+1];
    for (int i=0;i<=T.n; i++)
        O >> T.pd[i];
    return O; }
```

Лабораторный практикум

С использованием синтаксиса переопределения операций разработать стандартную арифметику объектов, включающую арифметические действия над объектами и целыми (вещественными, строками – в зависимости от вида объектов), присваивание, ввод и вывод в стандартные потоки, приведение к базовому типу данных, извлечение и обновление отдельных элементов (например, коэффициентов матрицы или символов строки). По возможности организовать операции в виде конвейера значений, с результатом – новым объектом и сохранением значений входных операндов. Для выбора варианта заданий использовать перечень классов из **10.1**.

Вопросы без ответов

Определите содержимое объектов после выполнения методов и переопределенных операций. Опишите схему их взаимодействия (копирование, отображение).

```
//-----103-03.cpp
```

```

// Общая часть класса для всех вариантов
class string{
    char *str;
    void load(char *s) { str=strdup(s); }
    void add(char *s) { str=(char*)realloc(str,strlen(str)+strlen(s)+1); strcat(str,s); }
    int find(char *s) { char *p=strstr(str,s); return p==NULL ? -1 : p-str; }
    int cmp(string &t) { return strcmp(str,t.str); }
public:
    string() { load(""); }
    string(char *s) { load(s); }
    string(string &t) { load(t.str); }
    ~string() { delete []str; }
    string &operator=(string &r) { delete str; load(r.str); return *this; }
//-----1
    char operator [](int n) { return n>=strlen(str) ? '?' : str[n]; }
    string &operator()(char c, int n0, int n1){
        for (int i=n0; i<=n1 && i<strlen(str); i++) str[i]=c;
        return *this; } };
    void main(){
        string s1("abcdefg"),s2("01234");
        s2(s1(s2[3],2,4)[0],1,3);
    }
//-----2
    string operator+(string &r) { string t(str); t.add(r.str); return t; }
    string operator()(int n0, int n1){
        if (n1 >= strlen(str)) n1=strlen(str)-1;
        char c=str[n1+1]; str[n1+1]='\0';
        string t(str+n0);
        str[n1+1]=c;
        return t;
    };
    void main(){
        string s1("abcdefg"),s2("01234"),s3;
        s3=s1(0,2)+s2(2,4);}
//-----3
    string operator+(string r) { r.add(str); return r; }
    };
    void main(){
        string s1("abc"),s2("012"),s3;
        s3=s1+s2+s2;}
//-----4
    friend string operator+(string one, string &two)
        { one.add(two.str); return one; }
    };
    void main(){
        string s1("abc"),s2("012"),s3;
        s3=s1+s2+s1; }
//-----5
    string &operator+(char *s) { add(s); return *this; }
    string &operator+(string &two) { add(two.str); return *this; }
    };
    void main(){
        string s1("ab"),s2("12"),s3;
        (s3=s1+s2+"zz")+s1;}
//-----6
    string operator+(char *s) { string two(s); two.add(str); return two; }
    string operator+(string two) { two.add(str); return two; }
    };
    void main(){
        string s1("abc"),s2("012"),s3;
        s3=s1+s2+"zz"+s2;}
//-----7
    string(char c, int n){
        str=new char[n+1];

```

```

        for (str[n--]='\0'; n>=0; n--) str[n]=c; }
string &operator()(char c, int n0, int n1){
    for (int i=n0; i<=n1 && i<strlen(str); i++) str[i]=c;
    return *this;
};
void main(){
    string s1('a',5),s2('b',5),s3("abcd");
    (s3=s1('b',0,2)('c',0,0))('d',4,10); }
//-----8
string &operator+(string &t) {
    for (int i=0; i<strlen(str) && i<strlen(t.str); i++)
        str[i]=t.str[i];
    return *this; }
string &operator+(char *s){ string t(s); return *this + t; }
};
void main(){
    string s1("abcdefg"),s2("123"),s3;
    (s3=s1+"qwerty")+s2; }
//-----9
string &operator()(int n0, int n1) {
    int k=strlen(str); n1++;
    while(n1<=k) str[n0++]=str[n1++];
    return *this; }
string &operator-(char *s) {
    int k=strlen(s), n0=find(s);
    if (n0!=-1) (*this)(n0,n0+k-1);
    return *this; }
};
void main(){
    string s1("abcdefg"),s2("1234"),s3;
    (s3=s1(2,4)-"bf")-"abc";
    s2-"12345"- "23";}
//-----10
string &operator()(int n0, int l) {
    int k=strlen(str);
    int n1=n0+l;
    while(n1<=k) str[n0++]=str[n1++];
    return *this; }
int operator[](char *s) { return find(s); }
};
void main(){
    string s1("abcdefg"),s2("12345678");
    s1(s1["def"],2);
    int n=s1["cf"];
    s2(s2["45"],2),s2(s2["36"],2); }

```